
GUÍA DE TALLER: CONTROL DE VERSIONES CON (GITHUB / GITLAB)

OBJETIVO

Aplicar control de versiones utilizando Git, comprendiendo su funcionamiento y uso práctico, e integrándolo con plataformas SaaS como GitHub o GitLab para gestionar código en la nube, incluyendo el flujo de trabajo colaborativo mediante Pull Requests.

REQUISITOS PREVIOS

- Tener instalado Git en el equipo
- Tener una cuenta en GitHub o GitLab
- Acceso a terminal (Git Bash, CMD o PowerShell)

PARTE 1: VERIFICACIÓN E INSTALACIÓN DE GIT

1. Verificar instalación:

```
git --version
```

Si no está instalado, descargar desde:

<https://git-scm.com/>

PARTE 2: CONFIGURACIÓN INICIAL

Configurar nombre de usuario:

```
git config --global user.name "Tu Nombre"
```

Configurar correo electrónico:

```
git config --global user.email "tu@email.com"
```

Ver configuración:

```
git config --list
```

PARTE 3: CREACIÓN DE REPOSITORIO LOCAL

1. Crear carpeta del proyecto:

```
mkdir mi_proyecto
```

```
cd mi_proyecto
```

2. Inicializar repositorio Git:

```
git init
```

Este comando crea un repositorio local donde se registrarán todos los cambios del proyecto.

PARTE 4: PRIMER ARCHIVO Y PRIMER COMMIT

1. Crear archivo:

```
echo "Hola mundo" > app.txt
```

2. Ver estado del repositorio:

```
git status
```

3. Agregar archivo al área de preparación:

```
git add app.txt
```

4. Crear primer commit:

```
git commit -m "Primer commit"
```

Un commit representa un punto en el historial del proyecto.

PARTE 5: MODIFICACIÓN Y CONTROL DE CAMBIOS

1. Modificar archivo:

```
echo "Nueva linea" >> app.txt
```

2. Ver diferencias:

```
git diff
```

3. Agregar cambios:

```
git add .
```

4. Crear nuevo commit:

```
git commit -m "Se agrega nueva linea"
```

5. Ver historial:

```
git log
```

PARTE 6: CREACIÓN DE REPOSITORIO EN LA NUBE (SaaS)

1. Ingresar a una plataforma u otra:

<https://github.com>

<https://gitlab.com>

2. Crear nuevo repositorio:

- Nombre: mi_proyecto
- No agregar README
- Crear repositorio

PARTE 7: CONEXIÓN ENTRE LOCAL Y NUBE

Copiar URL del repositorio y ejecutar:

```
git remote add origin https://github.com/usuario/mi_proyecto.git
```

Verificar conexión:

```
git remote -v
```

PARTE 8: SUBIR PROYECTO A LA NUBE

```
git branch -M main
```

```
git push -u origin main
```

Ahora el proyecto está disponible en la plataforma SaaS.

En caso de error:

```
git config --global --unset credential.helper
```

```
git push -u origin main
```

Esto borra las credenciales guardadas anteriormente y nos llevará a un login donde deberemos autenticarnos.

PARTE 9: TRABAJO CON RAMAS

1. Crear nueva rama:

```
git checkout -b nueva_funcionalidad
```

2. Modificar archivo:

```
echo "Cambio en rama" >> app.txt
```

3. Guardar cambios:

```
git add .  
git commit -m "Cambio en nueva funcionalidad"
```

4. Cambiar a rama principal:

```
git checkout main
```

5. Unir ramas:

```
git merge nueva_funcionalidad
```

Nota: este merge es local. Cuando el trabajo se hace con otra persona (por ejemplo, un alumno) sobre un repositorio compartido, la unión de ramas normalmente NO se hace con git merge local, sino a través de un Pull Request en GitHub, como se detalla en las Partes 10 y 11.

PARTE 10: TRABAJO COLABORATIVO — AGREGAR COLABORADORES Y PROTEGER MAIN

Sección nueva — necesaria para que los cambios de otra persona no se suban directo a main y queden siempre como Pull Request.

1. Agregar un colaborador al repositorio:

En GitHub: Settings → Collaborators → Add people, e ingresar el usuario o correo de la persona invitada.

2. Activar la protección de la rama main (recomendado antes de que el colaborador trabaje):

En GitHub: Settings → Branches → Add branch protection rule.

- En "Branch name pattern" escribir: main
- Marcar la opción "Require a pull request before merging"
- Guardar los cambios

Con esta protección activada, un push directo a main será rechazado por GitHub, incluso si el colaborador olvida crear una rama propia. Esto evita el problema de que los cambios queden aplicados directamente en main en vez de generarse como Pull Request.

PARTE 11: CREAR UN PULL REQUEST (FLUJO DEL COLABORADOR)

1. Clonar el repositorio:

```
git clone https://github.com/usuario/mi_proyecto.git
cd mi_proyecto
```

2. Crear y moverse a una rama nueva (paso indispensable — nunca trabajar directo sobre main):

```
git checkout -b feature/favicon
```

3. Realizar los cambios necesarios en el proyecto

(por ejemplo, agregar un archivo de favicon para la prueba).

4. Ver el estado y agregar los cambios:

```
git status
git add .
```

5. Crear el commit:

```
git commit -m "Agrega favicon de prueba"
```

6. Subir la rama (no main) al repositorio remoto:

```
git push origin feature/favicon
```

7. Crear el Pull Request:

- En GitHub, hacer clic en el botón "Compare & pull request" que aparece tras el push
- Completar título y descripción del cambio
- Hacer clic en "Create pull request"

8. Revisión y aprobación (rol del profesor/mantenedor del repositorio):

```
git fetch origin
git checkout feature/favicon
```

Revisar los cambios localmente o directamente en GitHub y, si están correctos, hacer clic en "Merge pull request".

Con la protección de rama de la Parte 10 activada, si el colaborador olvida el paso 2 y ejecuta git push origin main directamente, GitHub rechazará el push — el ejercicio se corrige solo.

PARTE 12: COMANDOS IMPORTANTES

git clone <url>	# Clonar un repositorio remoto
git status	# Ver estado
git add .	# Agregar cambios
git commit -m ""	# Guardar cambios
git log	# Ver historial
git diff	# Ver diferencias
git branch	# Ver ramas
git checkout	# Cambiar de rama
git checkout -b <nombre>	# Crear y cambiar a una rama nueva
git merge	# Unir ramas
git push	# Subir cambios
git pull	# Obtener cambios
git remote -v	# Ver repositorios remotos configurados